Arquitetura de Comunicação em Tempo Real via WebSocket

O OrbiVet utilizará uma arquitetura de comunicação em tempo real para permitir o envio de notificações relacionadas a eventos relevantes da plataforma, com destaque para a funcionalidade de animais desaparecidos.

O fluxo terá início quando um usuário realizar uma operação por meio do frontend, como registrar um possível avistamento. A requisição será encaminhada à API REST, responsável por validar os dados, verificar as permissões e aplicar as regras de negócio. Após a confirmação da persistência do registro no banco de dados, o backend poderá publicar um evento interno, como sighting.created.

O evento será encaminhado ao serviço de WebSocket, que identificará as conexões ativas dos destinatários autorizados e enviará uma mensagem em tempo real ao frontend correspondente. A interface poderá apresentar uma notificação ao tutor responsável pelo alerta, sem exigir que ele atualize manualmente a página.

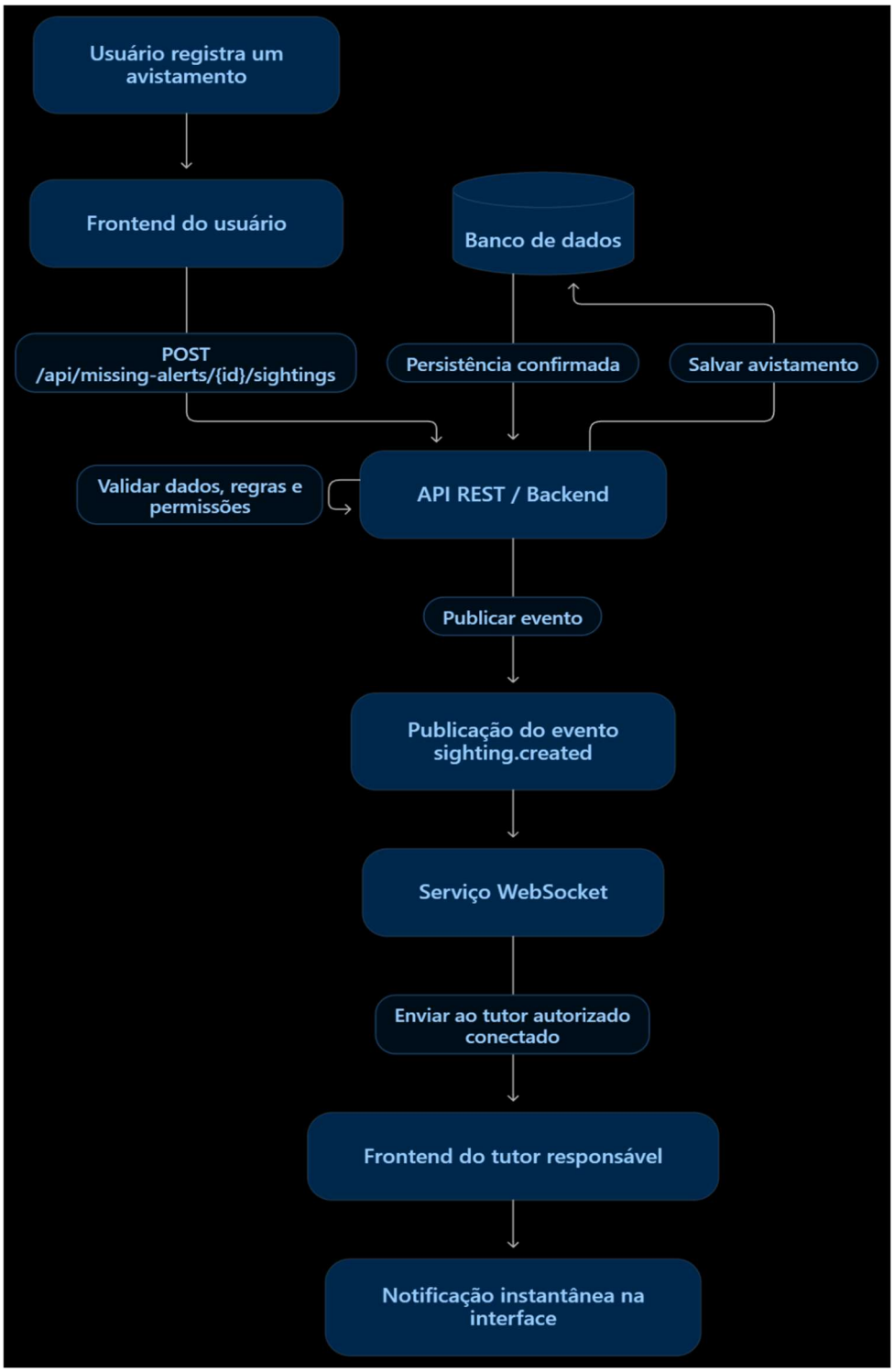
A conexão WebSocket será estabelecida por meio de um handshake HTTP e permitirá a troca bidirecional de mensagens enquanto permanecer ativa. O serviço deverá controlar a autenticação da conexão, a autorização dos destinatários e o encerramento ou a reconexão de clientes.

A utilização de WebSockets complementará a API REST, sem substituí-la. As operações de criação, consulta e atualização dos recursos continuarão sendo realizadas pela API. O WebSocket será utilizado para distribuir avisos de eventos ocorridos na aplicação.

Caso o destinatário esteja desconectado, o sistema deverá preservar a possibilidade de consultar as notificações posteriormente. Para isso, os registros de notificação deverão ser persistidos no banco de dados, e a interface poderá recuperá-los por meio da API REST após reconectar-se. O envio em tempo real será um mecanismo de comunicação imediata, não a única forma de acesso às informações.

Essa arquitetura busca reduzir a necessidade de consultas periódicas e melhorar a rapidez com que os responsáveis recebem informações relevantes sobre possíveis avistamentos e atualizações dos alertas de desaparecimento.

Na teoria como isso funcionaria?



Explicação:

- Um usuário registra um possível avistamento no frontend.

- O frontend envia uma requisição HTTP para a API REST.

- O backend valida os dados, verifica as permissões e aplica as regras de negócio.

- O registro do avistamento é salvo no banco de dados.

- Após a confirmação da persistência, o backend publica um evento interno.

- O serviço WebSocket identifica os destinatários conectados e autorizados.

- O tutor responsável recebe a notificação na interface.

E como isso funcionaria em nosso sistema?

1. Um usuário registra um possível avistamento de um animal perdido por meio do frontend do OrbiVet.

2. O frontend envia uma requisição HTTP POST para o endpoint /api/missing-alerts/{id}/sightings, encaminhando os dados do avistamento à API REST.

3. O backend recebe a requisição, valida os dados, verifica as permissões do usuário e aplica as regras de negócio necessárias.

4. Após a validação, o avistamento é salvo no banco de dados. O sistema também registra as notificações necessárias para permitir que sejam consultadas posteriormente, mesmo que o destinatário esteja desconectado.

5. O banco de dados confirma a persistência do avistamento. Essa confirmação permite que o backend prossiga com a publicação do evento.

6. O backend publica o evento interno sighting.created, indicando que um novo avistamento foi registrado com sucesso.

7. O evento é encaminhado ao serviço WebSocket, responsável pela comunicação em tempo real com os clientes conectados.

8. O serviço WebSocket verifica se o tutor responsável pelo alerta está conectado e autorizado a receber a notificação, respeitando as permissões de acesso e as regras de privacidade.

9. Se o tutor estiver conectado e autorizado, o serviço WebSocket envia a notificação ao frontend do tutor responsável, permitindo que a informação seja exibida instantaneamente na interface.

10. Se o tutor não estiver conectado, a notificação permanece disponível para consulta posterior, conforme o mecanismo de persistência representado no diagrama.

11. Quando o tutor voltar a acessar o sistema, o frontend poderá enviar uma requisição HTTP GET para o endpoint /api/notifications, solicitando as notificações disponíveis.

12. A API REST consulta as notificações armazenadas no banco de dados e devolve os resultados ao frontend, que poderá exibi-los ao tutor.

13. Caso ocorra um erro durante o processamento da requisição inicial, a API REST deverá retornar uma resposta HTTP apropriada, indicando sucesso ou falha da operação ao frontend.

